

IT UNIVERSITY OF COPENHAGEN

Master of Science in Computer Science &
Master of Science in Data Science

Hardware-Aware Benchmarking of Retrieval-Augmented Generation Systems

Submission Date: ??/??/2026

Authors

Jacob Sonne

Jesper Terkildsen

Supervisor

Ehsan Yousefzadeh-Asl-Miandoab

Email

json@itu.dk

jete@itu.dk

ehyo@itu.dk

Table of contents

1	Introduction	1
2	Related Works	2
	2.a RAG Architecture	2
3	Methodology	3
	3.a Datasets	3
	3.b Pipeline, Components & Setup	3
	3.c Component Metrics	4
4	Results	6
5	Discussion	7
6	Conclusion	7

Abstract

1 Introduction

The current Natural Language Processing (NLP) landscape is heavily based in the transformer architecture published by [AttenIsAllYouNeed] in 2017, and especially made mainstream after the release of GPT-3.5 [GPT3.5] in 2022 as one of the first Large Language Models (LLM's). These architectures have demonstrated a large degree of success on various NLP tasks, including sentiment analysis, summarization, question answering (QA), language translation, and code generation. Despite these diverse capabilities they naturally have a set of limitations as well. Outdated or static knowledge is the problem where the model is only informed up to a cut off date determined by its training data. Hallucination is the issue where the model generates plausible sentences from its training and input prompt, but the answer is not grounded in a source.

Both of these issues are addressed by Retrieval Augmented Generation (RAG) systems [RAG]. RAG is a framework around the LLM that gives it access to a search engine and a vector or graph based database. This makes it so the LLM does not entirely depend on its training data for generation but also the knowledge in the database, thus grounding its generation in a source and gives it access to more recent data.

In recent years the research literature on this topic has grown greatly as its potential has seen more wide spread adoption [recentRAGResearch].

2 Related Works

2.a RAG Architecture

Chunking

Embedding

VectorDB

Retriever

Reranker

LLM

3 Methodology

The following section will go into depth of the developed pipeline as whole, its individual components and the metrics implemented for each.

3.a Datasets

Wikipedia dump - psgs_w100, preindexed wikipedia dump with passage awareness [cite].

NQ - natural questions dataset filtered to only those with short and long answers.

3.b Pipeline, Components & Setup

The whole pipeline can be broken down into two distinct phases, the offline and on-line phase. The offline phase is the initial building of the knowledge-base, including chunking, embedding and indexing the raw text, in our case, from wikipedia. What we call the online phase is what a user would experience live from the system, this includes from the prompt submission into the embedder, retriever, re-ranker and finally generator which takes all the relevant ranked retrieved data in addition to user prompt and generates an output. The design of the pipeline allows for easy changing of each component in the pipeline, as well as component related parameters in one place. Additionally multiple pipelines can easily be set up for comparison of different setups one wishes to test against.

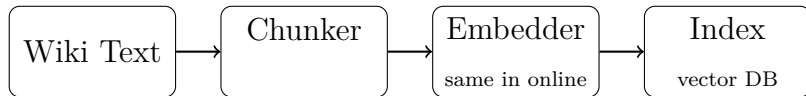


Figure 1: Offline RAG pipeline: chunking, embedding, and indexing

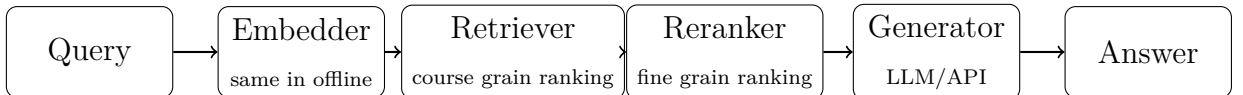


Figure 2: Online RAG pipeline: retrieval, reranking, and generation

3.c Component Metrics

Category	Metric	Definition	Description
Retrieval	Precision@K	$\frac{Rel_{doc}}{K}$	Fraction of top-K retrieved chunks that are relevant.
	Recall@K	$\frac{Rel_{doc}}{Rel_{AllDoc}}$	Fraction of all relevant chunks retrieved in top-K.
	F1@K	$2 \cdot \frac{P@K \cdot R@K}{P@K + R@K}$	Harmonic mean of Precision@K and Recall@K.
	MRR	$\frac{1}{ Q } \sum_{q \in Q} \frac{1}{rank_q}$	Rewards early appearance of the first relevant chunk.
	Hit Rate@K	$\mathbb{I}(\exists \text{ relevant in top-K})$	Whether at least one relevant chunk is retrieved.
Reranking	ΔMRR	$MRR_A - MRR_B$	Improvement in ranking after reranking.
Generation	BERTScore	$\sum \cos(e_{gen}, e_{ref})$	Embedding-based semantic similarity.
	ROUGE-1	$\frac{\# \text{ overlapping unigrams}}{\# \text{ unigrams in reference}}$	Word-level overlap with reference.
	Exact Match (EM)	$\mathbb{I}(y_{pred} = y_{ref})$	Exact string match with reference answer.
	None Ratio	$\frac{\#(\text{NONE outputs})}{ Q }$	Frequency of failed or null responses.

Table 1: RAG evaluation metrics with mathematical definitions

Rel_{doc} : relevant documents in top-K, Rel_{AllDoc} : total relevant documents, Q : set of queries.

Component	Hardware Metric
Memory	ram_used_mb, ram_total_mb, ram_avail_mb, ram_used_pct
CPU	cpu_util_pct
GPU	disk_util_pct, gpu_mem_total_mb, gpu_mem_free_mb, gpu_util_pct, dcgmi_power_w, dcgmi_gpu_util_pct, dcgmi_nvlink_tx_mbs, dcgmi_nvlink_rx_mbs, dcgmi_pcie_tx_mbs, dcgmi_pcie_rx_mbs
Disk I/O	disk_r_s, disk_w_s, disk_rkB_s, disk_wkB_s, disk_r_await_ms, disk_w_await_ms, disk_util_pct

Table 2: Hardware monitoring metrics.

4 Results

5 Discussion

6 Conclusion